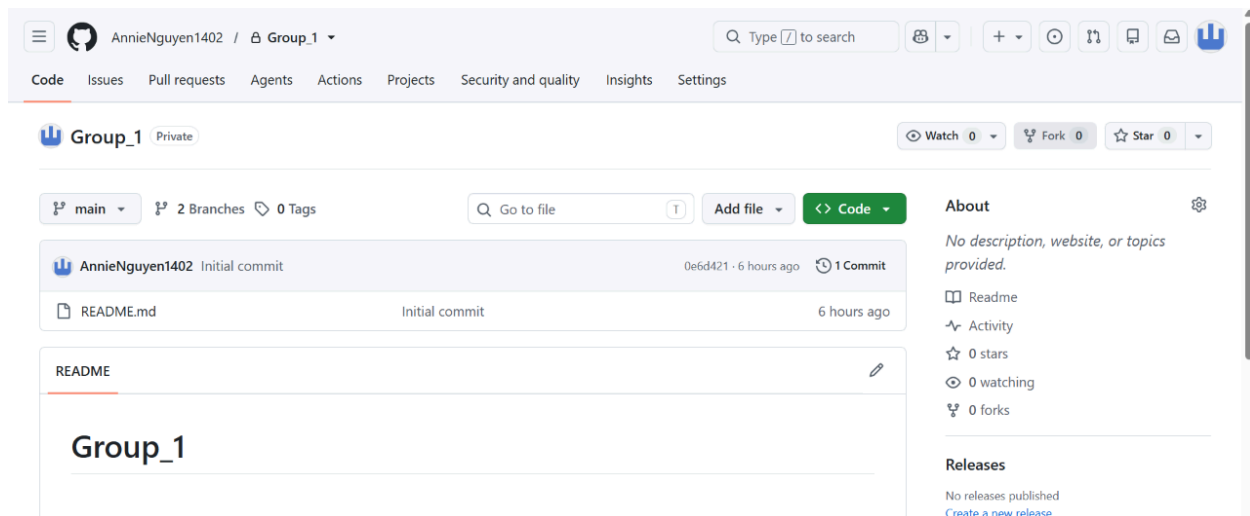
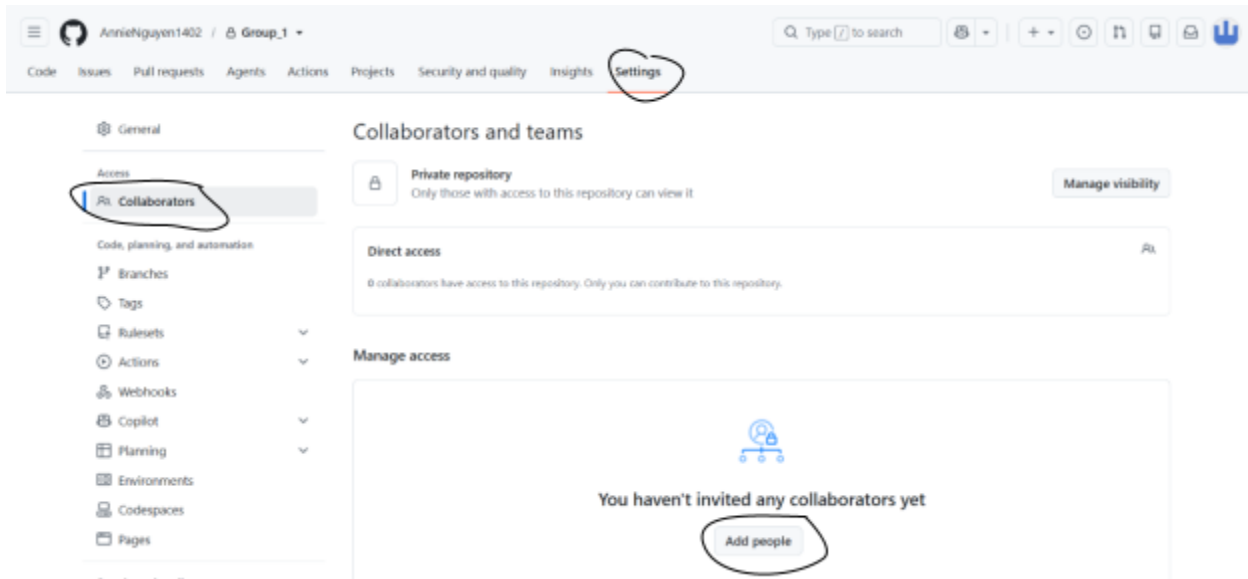


STANDARD GIT & GITHUB WORKFLOW FOR TEAMS

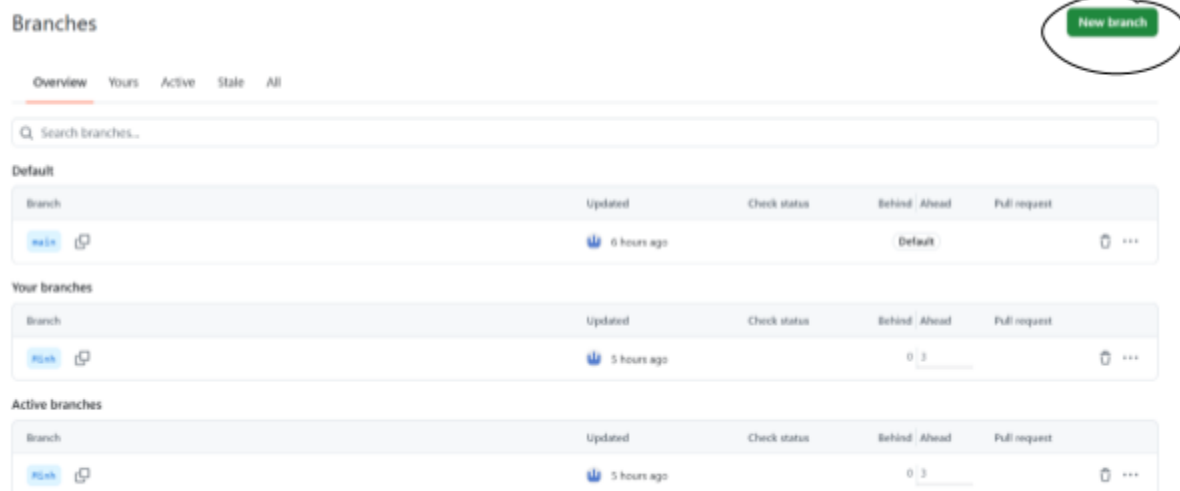
Part 1: Setting Up the Team Repository and Collaborators



- The team leader creates a new repository for the group. It is recommended to set this repository to Private so that only invited collaborators can view it.



- To add team members, navigate to **settings** -> **collaborators** -> **add people**.



- Each collaborator should create a new branch using their own name. Everyone will work on their respective branches before merging their changes into the **main** branch.

Part 2: Installation & Identity Setup (One-time only)

- After installing Git, the first step is to "declare your identity." Open your Terminal (Mac) or Git Bash/Command Prompt (Windows) and type:

```
git config --global user.name "Your name/Nickname"
git config --global user.email "Your email"
```

Note: You must use the exact email you used to register your GitHub account. If you enter it incorrectly or use a random email, GitHub will not record your contributions to the project.

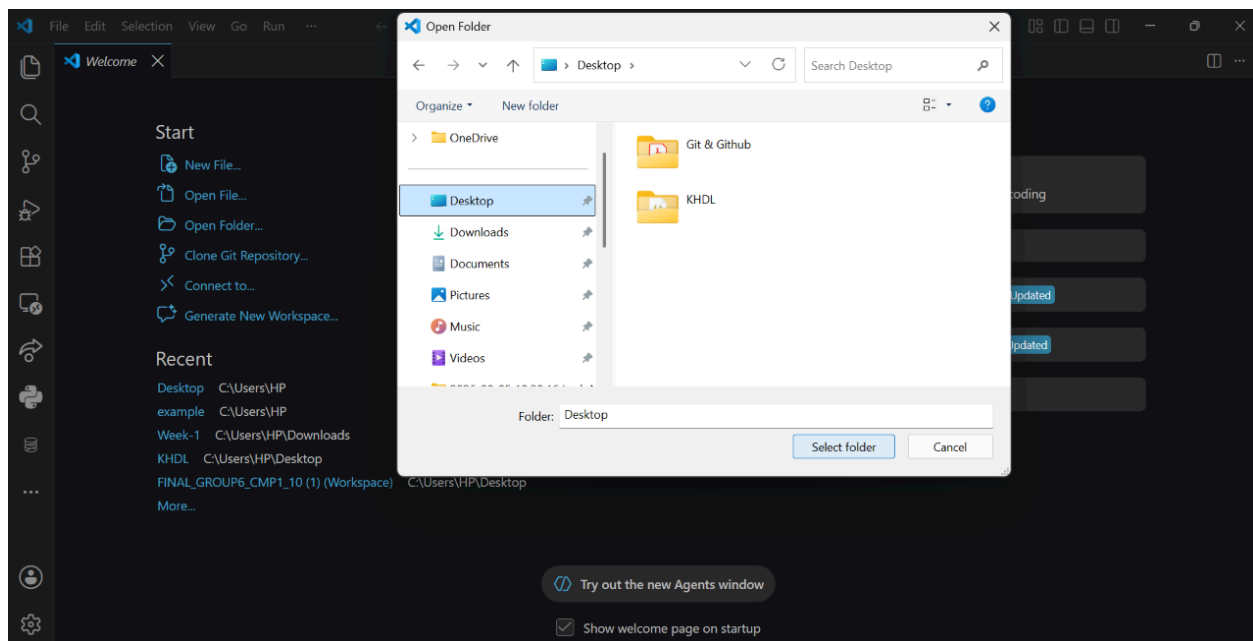
- Check identity list:

```
git config --global --list
```

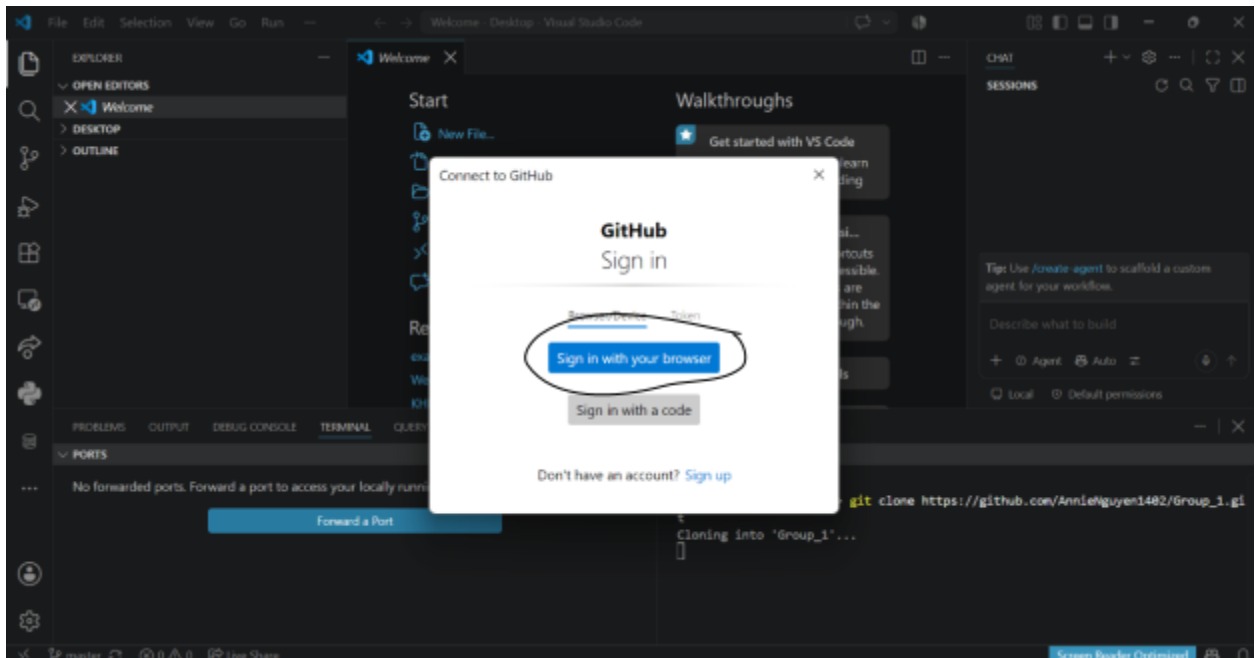
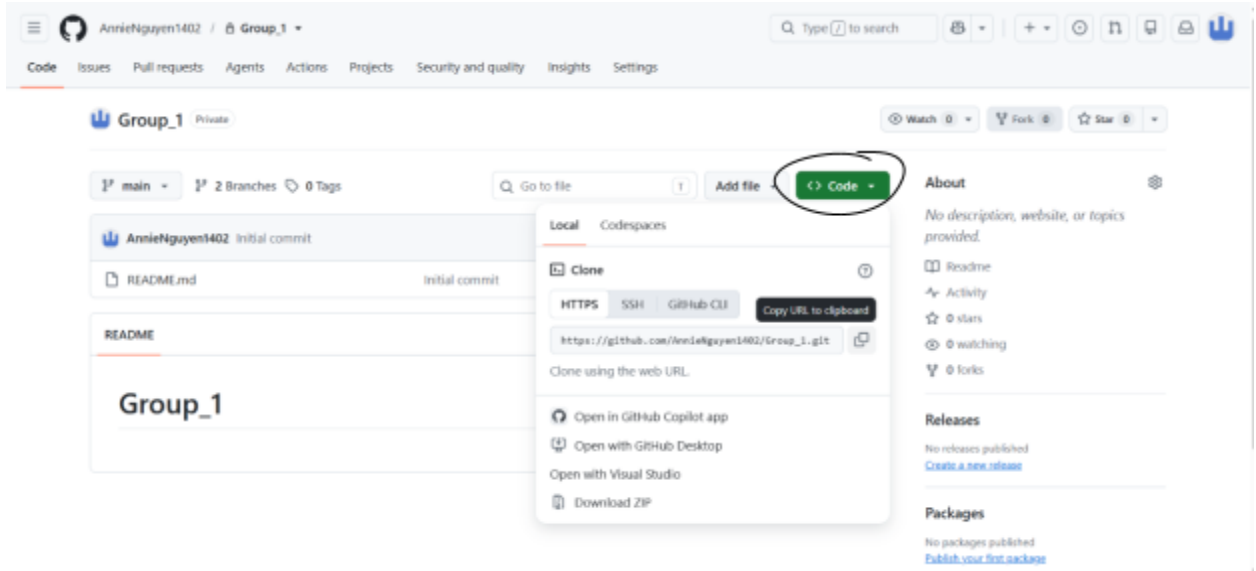
Part 3: Fetching The Project

When working with an existing project on the internet, **absolutely do not use the `git init` command**.

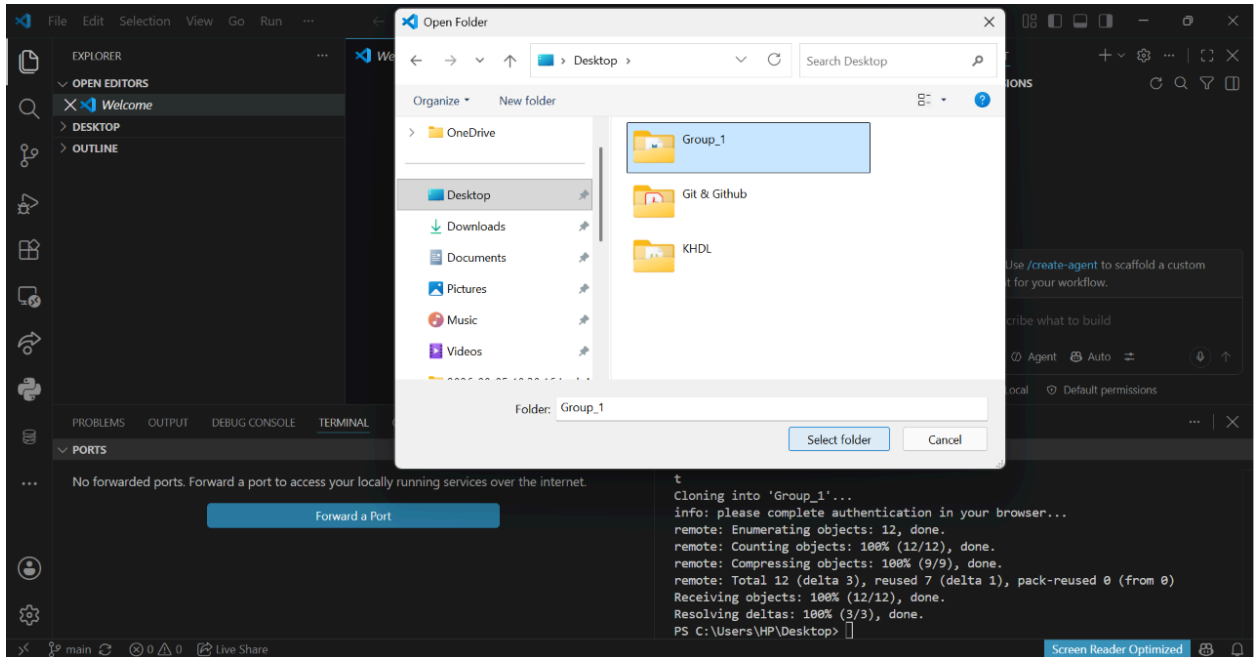
- Create a **completely empty** folder on your computer. **Choose** a direction that is familiar with you so you can easily find your folder (*Eg. I chose Desktop*).



- Open VS Code, select **Clone Repository** (or open the Terminal and type `git clone <Repo_URL>`).



- Open the folder you just created.



Part 4: Daily Workflow

Step 1: Pull the latest code from the main branch (MANDATORY BEFORE CODING)
You must ensure you are starting from the team's latest code base.

```
git checkout main  
git pull origin main
```

Note: You must ensure you are starting from the team's latest code base.

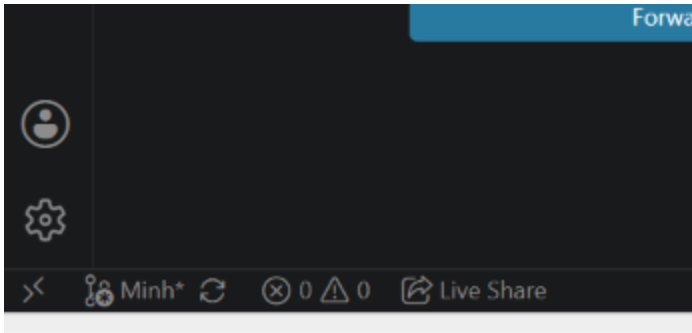
Step 2: Switch to a separate branch for your task. Every task/feature must be developed on its own separate branch.

```
git checkout ten_nhanh_cua_ban
```

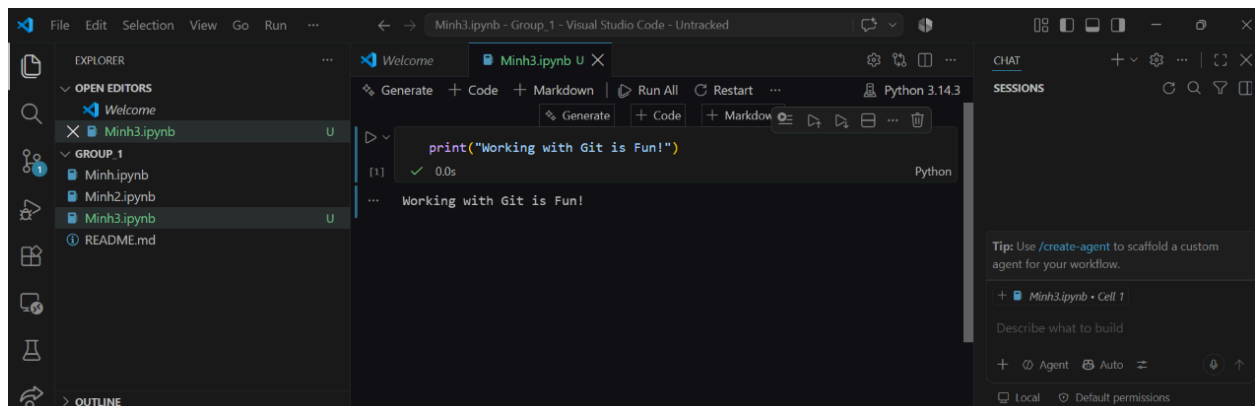
or

```
git switch ten_nhanh_cua_ban
```

Note: Forgetting to switch to your branch and coding directly on `main`. Always look at the bottom-left corner of VS Code to verify you are on your personal branch before typing any code.



Step 3: Write Code & Save changes



- Work as usual and press Save (Ctrl+S / Cmd+S).

Step 4: Add & Commit

Once done coding, you must "package" it.

```
git status
```

```
git add .
```

```
git commit -m "Write your comment here".
```

Note: Clearly state what file you worked on and its purpose. Recommended prefixes:

feat: for new features, **fix**: for bug fixes, **docs**: for documentation.

Step 5: Final Pull & Push code to GitHub

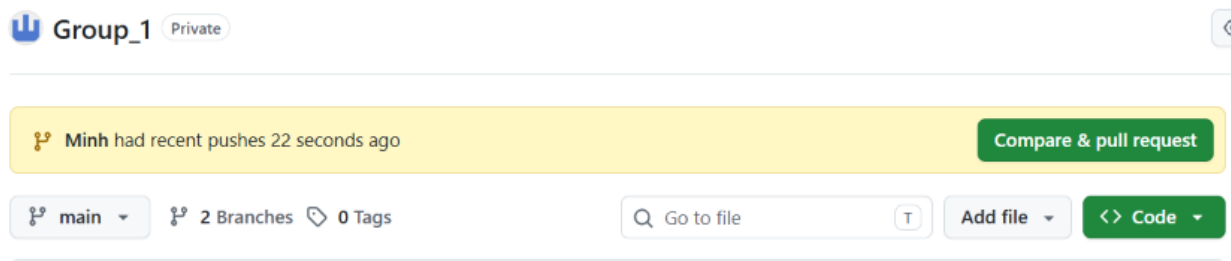
Before pushing to the internet, check if anyone has updated the main branch while you were working:

```
git pull origin main
```

- If everything goes smoothly: Push the code to your branch: `git push -u origin "Your branch"`
- If it shows red Conflicts: Move to Part 6 to resolve them.

Step 6: Create Pull Request & Merge Principles

Go to the GitHub website, find your branch, and click **Compare & pull request** to request merging your code into the **main** branch.



Note: The **main** branch is the "heart" of the project. When you Merge, the entire team will download that code during their next **pull**.

- **NO** creating a Pull Request & Merging if the task is incomplete.
- **NO** Merging if the code still has bugs to avoid crashing the project for everyone.
- **NO** clicking Merge on your own code. Let your teammates or Team Leader review it and click Approve/Merge.

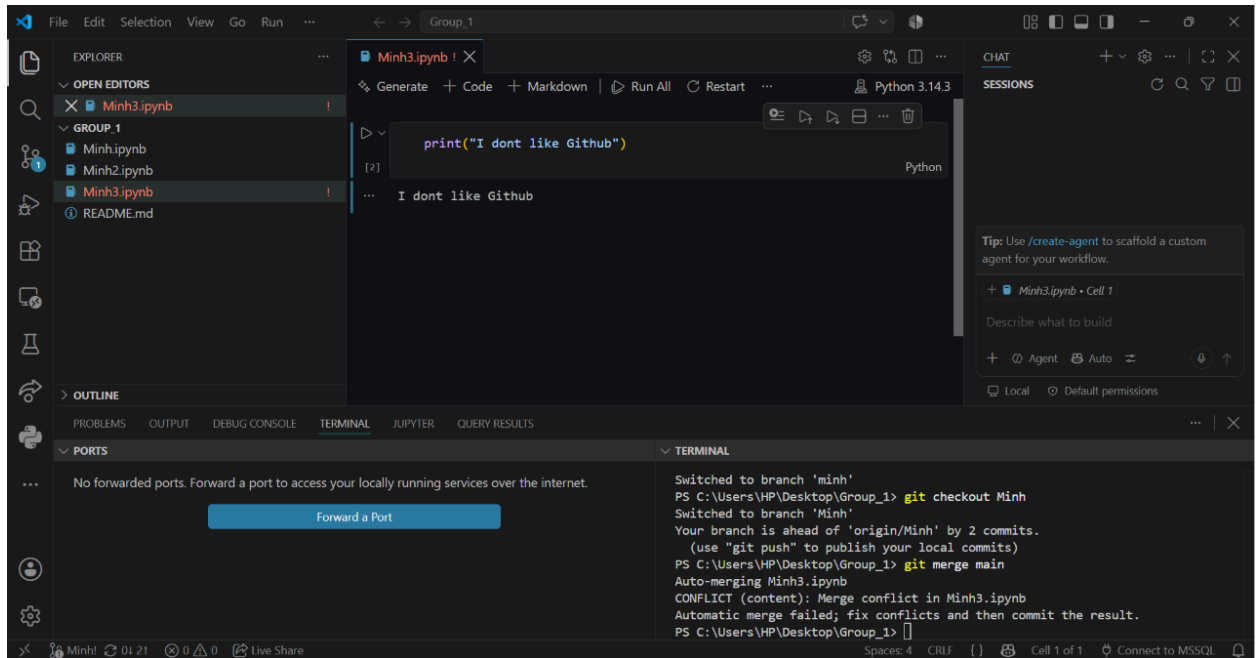
Part 5: How To Resolve CONFLICTS

Conflicts occur when two people edit the exact same line of code. Git needs you to manually specify whose code to keep.

Safe resolution steps (Only do this on your branch)

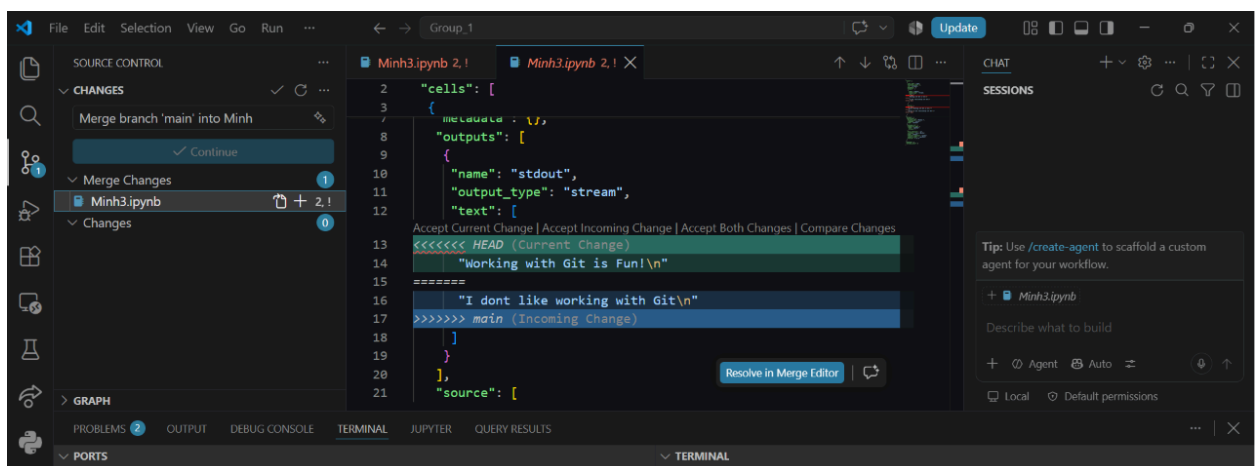
- While on your branch, pull the code from main to attempt a merge:

```
git pull origin main
```
- If a Conflict is reported, find the files highlighted in red.



- Select **Source control** -> The red exclamation mark (!) -> The raw JSON structure

When a conflict occurs, Git inserts raw conflict markers (`<<<<<< HEAD`, `=====`, `>>>>>> main`) directly into the file. Because this is a Jupyter Notebook, we are looking at the raw JSON code.



- VS Code will display 4 buttons above the conflicting code:

- **Accept Current Change:** Click this to keep *your* code from your current branch (the green `HEAD` block: `"Working with Git is Fun!\n"`). The other person's code will be discarded.
- **Accept Incoming Change:** Click this to keep your teammate's code that you just pulled from the `main` branch (the blue `main` block: `"I dont like working with Git\n"`). Your code will be discarded.
- **Accept Both Changes:** Click this to keep both lines of code.
Be very careful using this in a Jupyter Notebook (.ipynb). Because Notebooks use strict JSON formatting, blindly accepting both might break the syntax (e.g., missing commas or brackets), causing the file to corrupt and not open later.
- **Compare Changes:** Click this to open a split-screen view, allowing you to compare the two versions side-by-side before making a final decision.

- Save the file (Ctrl+S) after you have resolved ALL conflicting files.
- Finalize the resolution with these commands:

```
git add .
git commit -m "fix: Resolve conflict with main branch"
git push -u origin your_branch_name
```